

CSC3066: Deep Learning – Assignment 1
Michal Wisniewski
40373624
22/02/2026

Introduction

This report evaluates a manually implemented Artificial Neural Network trained using mini-batch gradient descent and backpropagation on the German Credit dataset. The model performance is analysed using training loss and test accuracy curves. Additional experiments investigate the effect of learning rate on accuracy.

Model Implementation

The experiments were conducted using the German Credit Simplified dataset, which contains 1000 instances, each with 24 numerical input features and a binary target label (0 = Bad Credit, 1 = Good Credit).

The dataset was randomly split into 80% training set and 20% test set. A fixed random seed (*random_state* = 42) was used to ensure reproducibility. Stratified sampling was applied during the split to preserve the original class distribution in both training and test sets.

Because the input features have different numerical scales, feature normalisation was applied using standardisation. Specifically, each feature was transformed to have zero mean and unit variance.

A fully connected Multi-Layer Perceptron (MLP) with one hidden layer was implemented from scratch using NumPy. The architecture is defined as follows:

Input layer: 24 neurons (corresponding to the 24 input features)

Hidden layer: 6 neurons, Sigmoid activation function

Output layer: 1 neuron, Sigmoid activation function

The model was trained using mini-batch gradient descent algorithm with a batch size of 32. Weights were randomly initialized from a uniform distribution between -0.1 and 0.1, while biases were initialized to zero. Binary cross-entropy loss was utilized to measure the discrepancy between the predicted probabilities and the actual target labels.

Model Evaluation and Analysis

The below figure shows the training loss and test set accuracy over 150 epochs. The training loss decreases consistently throughout training. It starts at around 0.61 and gradually declines to a final value of 0.53.

The loss curve shows a smooth downward trend, particularly during the first 60 epochs, after which the rate of decrease slows and begins to plateau. This indicates that the model is successfully minimising the binary cross-entropy objective and learning meaningful patterns from the training data.



The test accuracy is initially high (0.70), then gradually decreases and fluctuates, resulting in a final value of 0.645.

While the test accuracy does not increase over time, the continual decrease in training loss across epochs implies that the model is learning from the training data. The most significant reduction occurs in the earlier epochs (0-60), which suggests faster learning at the beginning of training, followed by slower convergence.

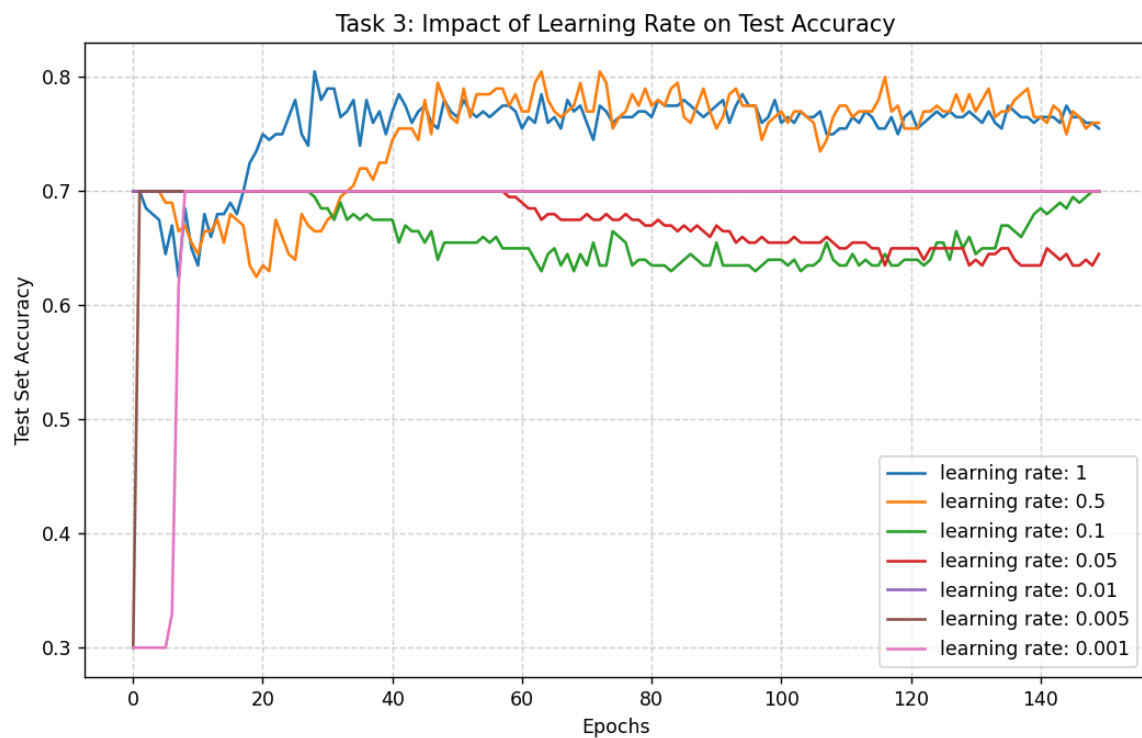
While training performance improves steadily, the test accuracy does not improve alongside it, rather decreasing gradually from 0.7 to 0.64, suggesting that the model is experiencing mild overfitting.

In regards to a critical discussion, the most noticeable issue with the final results in the relatively low final test accuracy of 0.64, as well as its gradual decline. This indicates that the model may not be generalising effectively to unseen data, perhaps due to the low number of neurons in the hidden layer (16) or the or the suboptimal the learning rate of 0.05. Another contributing factor may be the sigmoid activation function, which can affect the model adversely by causing vanishing gradients.

Hyperparameter Investigation

I chose to investigate the learning rate as it is the most critical hyperparameter in gradient descent. It controls the step size taken towards the minimum of the loss function. I tested the values: 1.0, 0.5, 0.1, 0.05, 0.01, 0.005, and 0.001. All other parameters (Hidden neurons = 16, Epochs = 150, Batch size = 32) remained constant to ensure a fair test.

The results of the Hyperparameter Investigation can be seen in the below graph:



High Learning Rates (1.0, 0.5): These rates showed the fastest initial "jump" in accuracy, reaching the 0.75–0.80 range within the first 20 to 40 epochs. However, the curves are highly volatile.

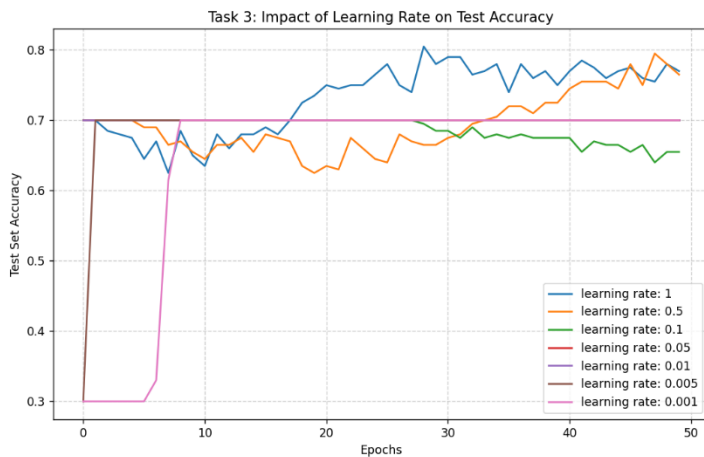
Medium Learning Rates (0.1, 0.05): These rates converged more slowly, struggling to maintain the initial 0.70 baseline, showing a downward trend before stabilizing or slightly recovering toward the end of the 150 epochs.

Low Learning Rates (0.01, 0.005, 0.001): These values resulted in almost flat lines at the 0.70 or 0.30 accuracy marks for much of the training. This suggests that the weight updates were too small to escape the initial state or overcome the vanishing gradient issues inherent in the Sigmoid activation function within the 150-epoch limit.

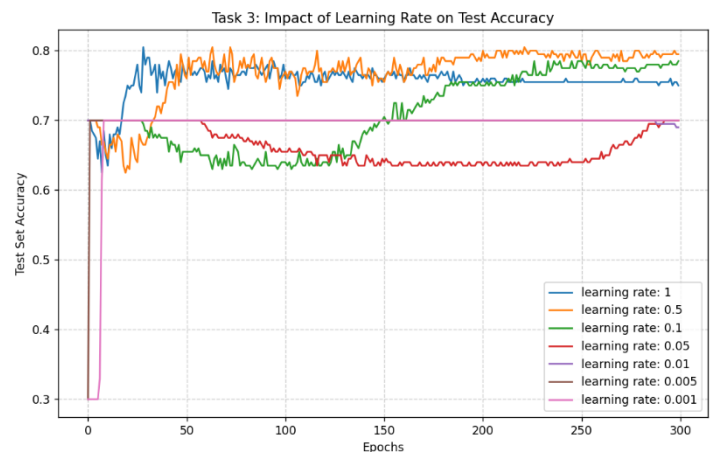
In this implementation, higher learning rates appear necessary to push the Sigmoid based weights effectively through the loss landscape; lower rates clearly lacked the momentum to reach the better-performing weight configurations within 150 epochs. With these results in mind, I would recommend using a learning rate of 0.5. While learning rate 1.0 reached a high accuracy faster than 0.5, learning rate 0.5 provided a slightly more stable trajectory and resulted in a similar final accuracy of 0.77.

However, when adjusting the learning rate, the optimal maximum number of epochs may also change. Due to this, I decided to alter the number of epochs to see what effects it may have on accuracy.

Epoch value 50:



Epoch value 300:



At 50 epochs the model is largely in a state of underfitting for most learning rates. Rates of 0.1 and 0.05 are still in a downward trend or haven't begun their climb, while the very low rates (0.01 to 0.001) are completely flat, indicating they haven't had enough time to move the weights away from their initial state.

At 300 epochs higher learning rates settle into a stable plateau, while moderate learning rates (0.1 and 0.05) finally begin to converge and catch up to the higher values, proving that they just required more time to navigate the loss landscape.

Given the results provided by 50 and 300 epochs, I would recommend a final learning rate value of 0.5. 50 epochs simply reinforces the previous analysis; 0.5 provided a slightly more stable trajectory and resulted in a similar final accuracy to 1.0. The findings provided by 300 epochs shows that learning rate 0.1 can catch up the higher learning rate values of 1.0 and 0.5, and while 1.0 retains a high accuracy, 0.5 is still the best learning rate. Learning rate 0.1 catches up to 0.5 at around epoch 200 but 0.5 produces a better accuracy throughout all 300 epochs. Learning rate 1.0 plateaus at around epoch 170, being overtaken by 0.5 in accuracy.